Formal Verification
Appunti

Alex "Morbidelli WhatsApp" Basta

Contents

Chapter 1 **Checking Transition Systems** Page

1.1 Transition Systems

1.2 Boolean Encoding of Finite-State Systems
Inductive Invariant checking with SAT — • Reachability checking —

Chapter 2 **Automating Resolution** Page

2.1 Formal Proof Systems
Derivations from Assumptions — • Consequence and Soundness in Propositional Logic — • Semantic vs. Syntactic Consequence —

2.2 Propositional Resolution
Decision and Simplification — • Resolution on Clauses — • Flatenning — • SAT Solvers —

2.3 FOL Resolution
NNF and Skolemization — • Instantiation and Resolution —

Chapter 3 **Quantifier Elimination with Presburger Arithmetic** Page

3.1 Verification Conditions and One-Point Rule

3.2 Sufficient Condition for Quantifier Elimination

3.3 DNF transformation

3.4 Eliminating Existentials from Conjunctions of PA Literals
Normalizing literals — • Exposing the variable: making the coefficient ±1 — • Lower and upper bounds — • Replacing the variable by test terms — • No lower bounds ($L = 0$) — • Summary of the algorithm and its consequences — • Improving efficiency in practice — • Complexity —

Chapter 4 **Generating Verification Conditions for Imperative Programs** Page

4.1 Programs and Specifications as Formulas

4.2 Rules for Simple Imperative Programs

4.3 Loop-Free Programs

4.4 Hoare Triples

Chapter 5 **Abstract Interpretation** Page

5.1 Programs as Control Flow Graphs

5.2 Approximating Sets

5.3 Partial Orders and Extreme Elements

5.4 Lattices

Chapter 1

Checking Transition Systems

1.1 Transition Systems

Definition 1.1.1: Transition System

A *transition system* is an n-uple $M = (S, I, r, A)$ where:

- S is the set of all states of the system
- $I \subseteq S$ is the set of initial states
- A is the set of all inputs
- $r \subseteq S \times A \times S$ is the transition relation (not necessarily deterministic)

We can define a set of error states $E \subseteq S$ and determine if the system can reach one of these or not.

Definition 1.1.2: Trace

A finite or infinite sequence $s_0, a_0, s_1, a_1, s_2...$ such that $\forall i. s_i \in S, a_i \in A$ and $s_0 \in I$.

If the trace is finite and finishes with s_n, it has length n.

$Traces(M)$ is the set of all traces of a system M.

Definition 1.1.3: Reachable States

A state $s_n \in S$ is *reachable* if there exists $t \in Traces(M)$ such that t ends with s_n.

$Reach(M) = \{s_n | \exists (s_0, a_0, ..., s_n) \in Traces(M)\}$

We can construct the state graph and explore it starting from the initial states, building $Reach(M)$.

Definition 1.1.4: Simplified Transition

We can define the simplyfied transition of a system M as

$$\overline{r} = \{(s, s') | \exists a \in A. (s, a, s') \in r\}$$

Note:

Even if r is deterministic, $\overline{r}$ can become non-deterministic.

Definition 1.1.5: Operations on relations

Let r_1, r_2 be relations, we can define:

- **Composition**

$$r_1 \circ r_2 \coloneqq \{(x,z) | \exists y.(x,y) \in r_1 \wedge (y,z) \in r_2\}$$

Iteration

$$r_1^0 = \Delta \coloneqq \{(x,x) | x \in S\} \quad r_1^{n+1} \coloneqq r_1 \circ r_1^n$$

Transitive Closure

$$r_1^* \coloneqq \bigcup_{n \geqslant 0} r_1^n$$

Image of a set under relation

$$r_1[X] \coloneqq \{y | \exists x \in X.(x,y) \in r_1\}$$

Theorem 1.1.1

$$Reach(M) = \vec{r}^*[I]$$

Definition 1.1.6: Post

Let $post(X) \coloneqq \overline{r}[X]$. We can now define

$$post^0(X) \coloneqq X \quad post^{n+1}(X) \coloneqq post(post^n(X))$$

Theorem 1.1.2

$$\bigcup_{n \geqslant 0} post^n(X) = Reach(M)$$

Definition 1.1.7: Invariant

Given a system M, $P \subseteq S$ is an *invariant* of M if

$$Reach(M) \subseteq P$$

Definition 1.1.8: Inductive Invariant

Given a system M, $P' \subseteq S$ is an *inductive invariant* of M if

- $I \subseteq P'$
- $\forall s \in S.(s,a,s') \in r \implies s' \in P'$

Note:

These two definitions aren't the same, since an invariant can contain a non-reachable state but not any of its (still non-reachable) neighburs. This would not be a valid inductive invariant.

Usually the set of states for a system (reachable, invariant) will be too large to store explicitly, so we can use formulas and data structures to represent them complactly. This allows us to use bounded model checking with SAT solvers instead of explicit-state exploration.

1.2 Boolean Encoding of Finite-State Systems

Definition 1.2.1: Boolean Encoding of a System

If we pick $n \geqslant log|S|, m \geqslant log|A|$ we can represent the system as follows:

- $S = \{0,1\}^n$, $A = \{0,1\}^m$
- I is the characteristic function $\{0,1\}^n \to \{0,1\}$
- $r = \{0,1\}^n \times \{0,1\}^n \times \{0,1\}^n$

To represent a boolean relation r, we could use a binary table. These are very fast, but require exponential space. Another option is to encode r as a propositional formula P with variables $s_1, ..., s_n, a_1, ..., a_m, s'_1, ..., s'_n$ such that

$$((v_1, ..., v_n), (u_1, ..., u_m), (v'_1, ..., v'_n)) \in r \iff P[s_i = v_i, a_j = u_j, s'_i = v'_i] \text{ is true}$$

This can be achieved by simply listing all the triples in r as a conjunction between the respective logical variables (negated if the bit is 0)

$$\bigvee_{((v_1,...,v_n),(u_1,...,u_m),(v'_1,...,v'_n))\in r} \left(\bigwedge s_i^{v_i} \wedge \bigwedge a_j^{u_j} \wedge \bigwedge {s'}_i^{v'_i} \right)$$

Formulas correspond to trees where variables are leaves and operations are nodes. We can use DAGs to avoid having to repeat parts of the tree, saving space in the representation. We can view DAGs as formulas with auxiliary variables that act as placeholders for parts of formulas.

1.2.1 Inductive Invariant checking with SAT

By representing I as a boolean formula in the same way, we can check if a given formula Inv is an inductive invariant by proving that $I \models Inv$ and $Inv, r \models Inv[\overline{s} := \overline{s'}]$. This can be achieved by flipping the problem and proving that it's impossible to satisfy the negations by using SAT solvers on the formulas

- $I \wedge \neg Inv$
- $Inv \wedge r \wedge \neg Inv[\overline{s} := \overline{s'}]$

1.2.2 Reachability checking

We construct a propositional formula T_j such that is satisfiable iff there exists a trace of length j that satisfies a certain error formula E. We use the notation p^i to denote the variable at step i.

$$T_j := Init[\overline{s} := \overline{s}^0] \wedge \left(\bigwedge_{i=0}^{j-1} r_i \right) \wedge E[\overline{s} := \overline{s}^j]$$

where $r^i := r[\overline{s}, \overline{a}, \overline{x}, \overline{s'} := \overline{s}^i, \overline{a}^i, \overline{x}^i, \overline{s'}^{i+1}]$

Chapter 2

Automating Resolution

2.1 Formal Proof Systems

We consider a set of logical formulas F (e.g. the formulas of propositional logic).

Definition 2.1.1: Proof system

A *proof system* is a pair (F, Infer) where $\mathit{Infer} \subseteq F^* \times F$ is a decidable set of *inference steps*. F^* denotes the set of all finite sequences with elements from F.

An inference step $((P_1, \ldots, P_n), C) \in \mathit{Infer}$ is written schematically as

$$\frac{P_1 \quad \ldots \quad P_n}{C}$$

and we say that from the *premises* $P_1, \ldots, P_n$ we derive the *conclusion* C. An inference step with $n = 0$ (no premises) is called an *axiom instance*.

Definition 2.1.2: Proof

Given a proof system (F, Infer), a *proof* is a finite sequence of inference steps $S_0, \ldots, S_m \in \mathit{Infer}$ such that for each S_i with $0 \leq i \leq m$ and for each premise P_j of S_i, there exists $0 \leq k < i$ such that P_j is the conclusion of S_k.

Index representation

Given this definition, each premise P_j can be replaced by the index k of the step S_k whose conclusion it is ($P_j \equiv \mathrm{Conc}(S_k)$). A proof is then a sequence of elements of $\{0, 1, \ldots\}^* \times F$, where each S_i has the form $(k_1, \ldots, k_n, C)$ with $0 \leq k_1, \ldots, k_n < i$ and

$$(\mathrm{Conc}(S_{k_1}), \ldots, \mathrm{Conc}(S_{k_n}), C) \in \mathit{Infer}.$$

Proofs as DAGs

Proofs can be viewed as directed acyclic graphs. For each step $(k_1, \ldots, k_n, C)$ in the sequence we introduce a node labelled by C, and directed labelled edges $(\mathrm{Conc}(S_{k_j}), j, C)$ for all premises $k_1, \ldots, k_n$.
To check such a proof, for each node we follow its incoming edges backwards, in the order of their indices, to find the premises, and then check that the resulting inference step belongs to *Infer*.

2.1.1 Derivations from Assumptions

Definition 2.1.3: Derivation

Given (F, Infer) with $\mathit{Infer} \subseteq F^* \times F$ and a set of assumptions $A \subseteq F$, a *derivation from* A in (F, Infer) is a proof in (F, Infer') where

$$\mathit{Infer}' = \mathit{Infer} \cup \{((), G) \mid G \in A\}.$$

Thus assumptions from A are treated just like axioms.

Definition 2.1.4: Provability

A formula $G \in F$ is *provable from assumptions* A, written $A \vdash_{\mathit{Infer}} G$, iff there exists a derivation from A in Infer containing an inference step whose conclusion is G. We write $\vdash_{\mathit{Infer}} G$ to denote that there is a proof in Infer with G as a conclusion; this is the same as $\emptyset \vdash_{\mathit{Infer}} G$.

2.1.2 Consequence and Soundness in Propositional Logic

Definition 2.1.5: Semantic consequence

Let $A \subseteq F$ be a set of propositional formulas and $C \in F$. Then C is a *semantic consequence* of A, written $A \models C$, iff for every environment e that defines all variables in $FV(C) \cup \bigcup_{P \in A} FV(P)$, if $[\![P]\!]_e = 1$ for all $P \in A$, then $[\![C]\!]_e = 1$.

Definition 2.1.6: Soundness

Given (F, Infer) with F propositional, an inference step $((P_1, \ldots, P_n), C) \in \mathit{Infer}$ is *sound* iff $\{P_1, \ldots, P_n\} \models C$. The proof system Infer is *sound* if every inference step in it is sound.

For axioms ($n = 0$), this reduces to saying that C is true under all interpretations, i.e. that C is a valid formula (tautology).

Theorem 2.1.1 Soundness

Let (F, Infer) be a proof system over propositional formulas. If every inference rule in Infer is sound, then $A \vdash_{\mathit{Infer}} G$ implies $A \models G$.

2.1.3 Semantic vs. Syntactic Consequence

- **Semantic consequence.** To check $\{A_1\} \models G$, we check all possible interpretations: in every propositional assignment (every "world" we can imagine), if A_1 is true then so is G. For propositional logic, a world assigns truth values (0 or 1) to the variables.
- **Syntactic consequence (provability).** To check $\{A_1\} \vdash G$, we look for a derivation of G with A_1 as an additional axiom. Syntactic consequence is specific to a particular proof system.

In general we have a set of assumptions $A = \{A_1, A_2, \ldots\}$. For a given proof system there are two possible inclusions:

- **Soundness:** if $A \vdash G$, then $A \models G$. Without this, the proof system is "broken".
- **Completeness:** if $A \models G$, then $A \vdash G$. Without this, there are true statements that cannot be proved, which may be acceptable.

2.2 Propositional Resolution

2.2.1 Decision and Simplification

Definition 2.2.1: Literal

A *propositional literal* is either a variable (e.g. x) or its negation ($\neg x$).

Definition 2.2.2: Clause

A *clause* is a disjunction of literals. Because $\vee$ is associative, commutative and idempotent, a clause can be represented as a finite set of literals. If C is a clause, then $[\![C]\!]_e = 1$ iff there exists a literal $L \in C$ such that $[\![L]\!]_e = 1$. The constant 0 is represented by the *empty clause* $\emptyset$.

As for any formulas, a finite set of clauses A can be interpreted as the conjunction of its elements. A set of clauses is therefore a formula in conjunctive normal form: for instance $\{\{a\}, \{b\}, \{\neg a, \neg b\}\}$ represents $a \wedge b \wedge (\neg a \vee \neg b)$.

Unit resolution

Definition 2.2.3: Unit clause and complement

A *unit clause* is a clause with precisely one literal, i.e. of the form $\{L\}$. The *complement* $\overline{L}$ of a literal L is defined by $\overline{x} = \neg x$ and $\overline{\neg x} = x$.

Definition 2.2.4: Unit resolution

Unit resolution is the special case of resolution in which one of the clauses is a unit clause:

$$\frac{C \qquad \{L\}}{C \setminus \{\overline{L}\}}$$

Soundness. If L is true, then $\overline{L}$ is false, so $\overline{L}$ can be deleted from the disjunction C.

Subsumption

Definition 2.2.5: Subsumption

When applying resolution, if we obtain a clause $C' \subseteq C$ that is a subset of a previously derived clause C, we can delete C and no longer consider it. Any use of C can be replaced by a use of C', with progress towards $\emptyset$ at least as good. We say that C' *subsumes* C.

In particular, if we derive a unit clause $\{L\}$, we can remove all other occurrences of L and $\overline{L}$. If $L \in C$ then C is subsumed by $\{L\}$, and if $\overline{L} \in C$ then C is subsumed by $C \setminus \{\overline{L}\}$.

2.2.2 Resolution on Clauses

Definition 2.2.6: Clausal resolution rule

The *clausal resolution* rule (a decision rule for clauses) resolves two clauses with respect to a variable x:

$$\frac{C_1 \cup \{x\} \qquad C_2 \cup \{\neg x\}}{C_1 \cup C_2}$$

Theorem (Soundness). Clausal resolution is sound: for all clauses C_1, C_2 and propositional variables x,

$$\{C_1 \cup \{x\},\ C_2 \cup \{\neg x\}\} \models C_1 \cup C_2.$$

Theorem 2.2.1 Refutational completeness

A finite set of clauses A is unsatisfiable if and only if there exists a derivation of the empty clause from A using clausal resolution.

Resolution as transitivity of implication

For formulas F_1, F_2, F_3, if $F_1 \to F_2$ and $F_2 \to F_3$ are true, so is $F_1 \to F_3$; thus $\to$ is a transitive relation on $\{0,1\}$. Resolution can be viewed as a consequence of this transitivity, using the fact that $P \to Q$ is equivalent to $\neg P \vee Q$: the clauses $C_1 \vee x$ and $C_2 \vee \neg x$ are equivalent to $(\neg C_1) \to x$ and $x \to C_2$, from which we obtain $(\neg C_1) \to C_2$, i.e. $C_1 \vee C_2$.

2.2.3 Flatenning

Converting formulas to clauses

To check a formula with resolution we must first turn it into a set of clauses, that is, a conjunctive normal form. The equivalences guaranteed to produce a conjunctive normal form are negation propagation (De Morgan's laws) together with distributivity of $\vee$ over $\wedge$:

$$\neg(F_1 \wedge F_2) \leftrightarrow (\neg F_1) \vee (\neg F_2), \qquad F_1 \vee (F_2 \wedge F_3) \leftrightarrow (F_1 \vee F_2) \wedge (F_1 \vee F_3).$$

In general this transformation is exponential: applying distributivity from left to right duplicates the subformula F_1, which may produce an exponentially larger formula.
If we were willing to pay this cost, we might as well transform into disjunctive normal form, because satisfiability of a disjunction of conjunctions $D_1 \vee \ldots \vee D_n$ is trivial to check:

$$\exists e.\, [\![D_1 \vee \ldots \vee D_n]\!]_e = 1 \iff (\exists e.\, [\![D_1]\!]_e = 1) \vee \ldots \vee (\exists e.\, [\![D_n]\!]_e = 1),$$

and each conjunction D_i is checked separately. Hence, when the conversion is exponential, converting to CNF is no better than converting to DNF for checking satisfiability. This motivates a cheaper transformation that preserves only satisfiability.

Equivalence and equisatisfiability

Definition 2.2.7: Equivalence

Formulas F_1 and F_2 are *equivalent* iff $F_1 \models F_2$ and $F_2 \models F_1$, i.e. $\forall e.\, [\![F_1]\!]_e = [\![F_2]\!]_e$. A transformation producing an equivalent formula is *equivalence preserving*.

Definition 2.2.8: Equisatisfiability

Formulas F_1 and F_2 are *equisatisfiable* iff F_1 is satisfiable whenever F_2 is satisfiable, and vice versa. A transformation producing an equisatisfiable formula is *satisfiability preserving*.

Equivalent formulas are always equisatisfiable, but the converse does not hold in general: two different satisfiable formulas are equisatisfiable. An equisatisfiable formula may use additional variables, and not every assignment of the original formula is a model of the new one, but every model of the original can be extended to a model of the new formula.

Flattening

Definition 2.2.9: Flattening

Let F be a formula, G another formula, and $x \notin FV(F)$ a propositional variable. Let $F[G := x]$ denote the result of replacing an occurrence of the subformula G inside F with x. Then F is equisatisfiable with

$$(x \leftrightarrow G) \wedge F[G := x].$$

This satisfiability-preserving transformation is called *flattening*: a subformula is picked and given a name by a fresh variable.

Proof of equisatisfiability. A satisfying assignment of the new formula is also a satisfying assignment of the old one. Conversely, since x does not occur in F, if $[\![F]\!]_e = 1$ we can redefine $e(x)$ to be $[\![G]\!]_e$, which makes the new formula true.
Flattening works in any formalism that supports equality (here: equivalence). The strategy is to apply it starting from the smallest non-variable subformulas.

Tseytin's transformation

Tseytin's transformation converts a formula with $\neg, \wedge, \vee, \rightarrow, =, \oplus$ into an equisatisfiable set of clauses in polynomial time:

1. Replace $F_1 \rightarrow F_2$ with $\neg F_1 \vee F_2$, and push negations into the propositional variables using De Morgan's laws and by switching between $\oplus$ and $=$.
2. Repeatedly flatten an occurrence of a binary connective whose arguments are literals.
3. In the resulting conjunction, express each equivalence as a conjunction of clauses. For example,

$$x = (a \wedge b) \rightsquigarrow \{\neg x, a\},\ \{\neg x, b\},\ \{\neg a, \neg b, x\},$$
$$x = (a \vee b) \rightsquigarrow \{\neg x, a, b\},\ \{x, \neg a\},\ \{x, \neg b\},$$

   and similarly for $=$ and $\oplus$.

(See also Section 1.7.3 of *The Calculus of Computation*.)

2.2.4 SAT Solvers

A SAT solver takes as input a set of clauses. To check satisfiability of a formula, convert it to an equisatisfiable set of clauses in polynomial time using Tseytin's transformation. To check validity of a formula, take its negation, check satisfiability, and negate the answer.
To check satisfiability of a set of clauses there are two basic approaches:

- **Resolution on clauses**, favoring unit resolution and applying subsumption (complete; Davis and Putnam, 1960).
- **Truth table method:** try one value of a variable, then the other (space efficient).

The DPLL algorithm

The Davis–Putnam–Logemann–Loveland (DPLL) algorithm combines these ideas. Given a set of clauses S, it first applies unit propagation (boolean constraint propagation) and subsumption, obtaining S_1:

- if $\emptyset \in S_1$, the set is unsatisfiable;
- if S_1 contains only unit clauses, it is satisfiable and S_1 gives a satisfying assignment;
- otherwise, instead of doing general resolution, it creates more unit literals: it picks a literal L occurring in a clause of S_1 with $\{L\} \notin S_1$ and recursively solves $S_1 \cup \{\{L\}\}$ and $S_1 \cup \{\{\overline{L}\}\}$; the set is satisfiable if either call succeeds.

Unit propagation repeatedly takes a unit clause $\{U\}$ and a clause C with $\overline{U} \in C$, and replaces C by $C \setminus \{\overline{U}\}$. Subsumption repeatedly deletes a clause C_2 whenever another clause $C_1 \subseteq C_2$ is present.

Data structures

The simple algorithm above generates new clauses in unit propagation and deletes clauses in unit propagation and subsumption, which is very inefficient. SAT solvers use better data structures instead:

- All unit clauses are represented as a *current assignment*: a partial map (environment) e from some of the variables to truth values, initially empty. The unit clause $\{\neg a\}$ becomes $e(a) = 0$ and $\{a\}$ becomes $e(a) = 1$.
- Whenever a new literal L becomes true, the solver checks whether e already assigns the variable in a contradictory way; if so, a *conflict* is detected, which corresponds to $\emptyset$.
- Instead of resolving $\{L_1, L_2, \ldots, L_n\}$ with a unit literal, each clause is interpreted in the context of the current e: once $[\![L_1]\!]_e = 0$, the clause is read as $\{L_2, \ldots, L_n\}$.
- When all literals of a clause are 0 except one literal, say a variable p, then p must be true, so the assignment is updated to $e' = e \cup \{(p, 1)\}$, reporting a contradiction if already $e(p) = 0$. The *two-watched-literal* data structure makes detecting this situation fast.
- Instead of subsumption, clauses that are true in the current e are marked and ignored.

Clause learning and proofs

Definition 2.2.10: CDCL

Conflict-driven clause learning (CDCL) maintains the progress of exploring the search space using *learned clauses*. Each learned clause is derived by resolution from existing ones. For instance, if decisions setting literals L_1, L_2, L_3 to true lead to a conflict, then $\neg(L_1 \wedge L_2 \wedge L_3)$ is a resolution consequence of the input formula and can be added to the clause set.

Given the subtlety of SAT solvers, one uses proof representations (e.g. DRAT) that can be checked independently by simple and efficient proof checkers, which run in polynomial time.

Limits of resolution

There are combinatorial statements (e.g. the pigeonhole principle) that give an infinite family of unsatisfiable formulas $F_1, F_2, \ldots$ such that the shortest resolution proof $F_i \vdash \emptyset$ is exponential in the size of F_i (Urquhart, 1987). No such hard cases are known for *extended resolution*, which can introduce fresh propositional variables that name given clauses (Audemard, Katsirelos, Simon, AAAI-10). An efficiently checkable proof system with short proofs for all unsatisfiable formulas would imply NP = coNP.

2.3 FOL Resolution

Automating first-order logic

First-order logic (FOL) supports arbitrary (uninterpreted) predicates and functions, whose meaning is constrained through axioms. FOL can be used in practice to formalize most of mathematics (e.g. through set theory axioms), and thus all program verification problems.
To prove that a property holds, we describe the property by a formula F, describe the functions and relations in F by a sequence of axioms S, and check whether the sequence $(\neg F; S)$ is contradictory. If it is, then F follows from S.
There is a procedure that, if F does follow from S, establishes this in finite time (we do not know how long it will take, and if F does not follow, the procedure may loop). Resolution for FOL is one such procedure.

Syntax

Definition 2.3.1: First-order signature

A first-order signature L (also called a *language*) specifies a countable set of function symbols f and predicate symbols p. Constants c are function symbols taking no arguments. The arity ar of a function or predicate symbol is its number of arguments.

Formulas F and terms t are given by

$$F ::= p(t_1, \ldots, t_n) \mid \forall x.\, F \mid \exists x.\, F \mid \top \mid \bot \mid \neg F \mid F_1 \wedge F_2 \mid F_1 \vee F_2 \mid F_1 \to F_2 \mid F_1 \leftrightarrow F_2$$
$$t ::= x \mid c \mid f(t_1, \ldots, t_n)$$

where $x \in \mathit{Var}$ ranges over a countably infinite fixed set of variables. If $ar(f) = 2$ we may form the term $f(t_1, t_2)$, and if $ar(p) = 2$ we may form the formula $p(t_1, t_2)$. Quantifiers range only over variables, not over function or predicate symbols; this is what makes the logic first-order.

Definition 2.3.2: Atomic formula, literal, clause

An *atomic formula* is $p(t_1, \ldots, t_n)$ (it contains no logical connectives or quantifiers). A *literal* is an atomic formula or its negation $\neg p(t_1, \ldots, t_n)$. A *clause* is a disjunction of literals.

Semantics

Definition 2.3.3: First-order interpretation

A *first-order interpretation* is $I = (D, e)$ where $D \neq \emptyset$ (finite or infinite) and e maps

- each predicate symbol p with $ar(p) = n$ to $e(p) : D^n \to \{0, 1\}$,
- each function symbol f with $ar(f) = n$ to a total function $e(f) : D^n \to D$ (each constant c to an element $e(c) \in D$),
- each variable x to an element $e(x) \in D$.

The truth value $[\![F]\!]_I \in \{0, 1\}$ is defined by recursion on the formula tree with the expected rules, for example:

$$[\![F_1 \wedge F_2]\!]_I = [\![F_1]\!]_I \wedge [\![F_2]\!]_I, \qquad [\![\neg F]\!]_I = \neg [\![F]\!]_I, \qquad [\![\bot]\!]_I = 0,$$
$$[\![f(t_1, \ldots, t_n)]\!]_I = e(f)([\![t_1]\!]_I, \ldots, [\![t_n]\!]_I), \qquad [\![p(t_1, \ldots, t_n)]\!]_I = e(p)([\![t_1]\!]_I, \ldots, [\![t_n]\!]_I), \qquad [\![x]\!]_I = e(x),$$

$$[\![\forall x.\, F]\!]_{(D,e)} = 1 \text{ iff } \forall d \in D.\ [\![F]\!]_{(D, e[x:=d])} = 1, \qquad [\![\exists x.\, F]\!]_{(D,e)} = 1 \text{ iff } \exists d \in D.\ [\![F]\!]_{(D, e[x:=d])} = 1,$$

where $e[x := d](y) = d$ if $x = y$ and $e(y)$ otherwise. The meaning of function and relation symbols is fixed in e, so quantification over them (second-order or higher-order logic) is not expressible.

Definition 2.3.4: Validity and satisfiability

A formula F is *valid* if $[\![F]\!]_{(D,e)} = 1$ for all interpretations (D, e), for arbitrarily small or large sets D and all choices of e. A formula F is *satisfiable* if there exists an interpretation (D, e) such that $[\![F]\!]_{(D,e)} = 1$.

Satisfiability and validity are statements in the metatheory: the domain, functions and relations are all existentially quantified when asking about satisfiability, and all universally quantified when asking about validity. F is valid if and only if $\neg F$ is not satisfiable. To check validity of F we therefore check satisfiability of $\neg F$, aiming for a negative answer, after transforming $\neg F$ into a normal form.

2.3.1 NNF and Skolemization

Negation normal form

If $F \leftrightarrow G$ is a valid FOL formula, then inside any formula H we can replace a subformula F by G without changing the truth value: $H[F] \rightsquigarrow H[G]$. Using this we transform formulas to negation normal form with the rewrites

$$F_1 \leftrightarrow F_2 \rightsquigarrow (F_1 \to F_2) \wedge (F_2 \to F_1), \qquad F_1 \to F_2 \rightsquigarrow \neg F_1 \vee F_2, \qquad \neg\neg F \rightsquigarrow F,$$
$$\neg(F_1 \wedge F_2) \rightsquigarrow \neg F_1 \vee \neg F_2, \qquad \neg(F_1 \vee F_2) \rightsquigarrow \neg F_1 \wedge \neg F_2,$$
$$\neg\forall x.\, F \rightsquigarrow \exists x.\, \neg F, \qquad \neg\exists x.\, F \rightsquigarrow \forall x.\, \neg F,$$
$$\neg\bot \rightsquigarrow \top, \qquad \neg\top \rightsquigarrow \bot.$$

Definition 2.3.5: Negation normal form (NNF)

A formula is in *negation normal form* if negation is applied only to atomic formulas and the only other propositional connectives are $\wedge$ and $\vee$.

Skolemization

Definition 2.3.6: Skolem function

Replacing an existential quantifier by a function symbol is justified as follows. For a fresh function symbol s, the formula $p(x, s(x)) \to \exists y.\, p(x, y)$ is valid (take y to be $s(x)$), so if $\forall x.\, p(x, s(x))$ is satisfiable, so is $\forall x.\, \exists y.\, p(x, y)$. Conversely, given an interpretation in which $\forall x.\, \exists y.\, p(x, y)$ holds, we construct (by the axiom of choice) a function $\bar{s}$ assigning to each $x_d \in D$ some y_d with $e(p)(x_d, y_d) = 1$, and extend the interpretation with $e' = e[s := \bar{s}]$, in which $\forall x.\, p(x, s(x))$ holds. A new function symbol s used in this way is a *Skolem function* (after Thoralf Skolem). The same works for any quantifier-free formula in place of p, and the transformation preserves satisfiability.

Definition 2.3.7: Skolemization

In a formula in negation normal form, *Skolemization* replaces a subformula

$$\exists y.\, F(x_1, \ldots, x_n, y)$$

by

$$F(x_1, \ldots, x_n, g(x_1, \ldots, x_n)),$$

where g is a new function symbol of arity n (a Skolem function) and $\{x_1, \ldots, x_n\}$ are the free variables of the subformula, computed by

$$FV(c) = \emptyset, \qquad FV(x) = \{x\}, \qquad FV(f(t_1, \ldots, t_n)) = FV(t_1) \cup \ldots \cup FV(t_n) = FV(p(t_1, \ldots, t_n)),$$
$$FV(F_1 \wedge F_2) = FV(F_1) \cup FV(F_2), \quad FV(\neg F) = FV(F), \qquad FV(\forall x.\, F) = FV(F) \setminus \{x\} = FV(\exists x.\, F).$$

For quantifiers $Q_1 x_1. \ldots Q_n x_n.\, F$ we eliminate existentials starting from the outside (left), and the arguments of the Skolem function for $\exists x_i$ are only the universally quantified variables among $x_1, \ldots, x_{i-1}$.

Prenex and conjunctive normal form

Bound variables can be renamed while preserving equivalence, and a quantifier can be pulled in or out of a formula in which the variable does not occur:

- $(\forall x.\, F) \leftrightarrow \forall y.\, F[x := y]$, where y is fresh;
- $(\forall x.\, F) \vee G \leftrightarrow \forall x.\, (F \vee G)$;
- $(\forall x.\, F) \wedge G \leftrightarrow \forall x.\, (F \wedge G)$.

We can thus move quantifiers outwards.

Definition 2.3.8: Prenex normal form

A formula is in *prenex normal form* if it has the form $Q_1 x_1 . Q_2 x_2 . \ldots Q_n x_n . G$ where $Q_i \in \{\forall, \exists\}$ and G has no quantifiers.

The quantifier-free part G can then be transformed into conjunctive normal form as in propositional logic, using $(F \wedge G) \vee H \leftrightarrow (F \vee H) \wedge (G \vee H)$. As before, fresh predicate symbols can be introduced to avoid exponential blow-up.

Resulting conjunction of clauses

Theorem. Each formula F can be transformed into an equisatisfiable formula of the form

$$(\forall x_1, \ldots, x_n . C_1) \wedge \ldots \wedge (\forall x_1, \ldots, x_n . C_m)$$

where each C_i is quantifier-free and, moreover, a disjunction of first-order literals.
Indeed, we move negations to the predicates (NNF), eliminate $\exists$ by Skolemization, and pull $\forall$ and $\wedge$ to the outside, leaving $\vee$ applied to literals.

Definition 2.3.9: First-order clause

Each C_i above is a *first-order clause*: a disjunction of first-order literals, where a literal is an atomic formula $p(t_1, \ldots, t_k)$ or its negation. A clause can also be viewed as a set of literals, because the order of disjuncts does not matter up to equivalence; the empty clause means $\bot$. For a formula F, the set of clauses obtained is denoted $\mathit{clauses}(F)$.

Universal quantifiers are omitted because all variables are universally quantified. By convention, variables are denoted $x, y, z, \ldots$ and constants $a, b, c, \ldots$.

Sets of formulas

A set of formulas S is true in an interpretation I if every formula in it is true:

$$[\![S]\!]_I = 1 \iff \forall F \in S . \; [\![F]\!]_I = 1.$$

For finite S this equals the truth value of the conjunction of the elements of S. Two sets are *equivalent* iff the sets of interpretations in which they are true are equal, and S is *satisfiable* iff there is an I with $[\![S]\!]_I = 1$. Consequently, F is satisfiable iff $\mathit{clauses}(F)$ is satisfiable.

2.3.2 Instantiation and Resolution

Sound inference rules and instantiation

Definition 2.3.10: Sound inference rule

An inference rule

$$\frac{F_1 \quad \ldots \quad F_n}{G}$$

is *sound* iff the formula $(F_1 \wedge \ldots \wedge F_n) \rightarrow G$ is valid.

If $(*)$ is a sound inference rule and $F_1, \ldots, F_n \in S$, then S and $S \cup \{G\}$ are equivalent. Using the convention that variables are universally quantified, the *instantiation* rule

$$\frac{C}{C[x := t]}$$

is sound, where $C[x := t]$ denotes substituting variables of C by terms. In general the terms t may contain other variables. Special cases are:

- **renaming:** replace variables with fresh variables;
- **ground instantiation:** replace variables with ground terms, which have no variables (they are built only from constants and function symbols).

Resolution for FOL clauses

The following transitivity of universal implication is valid for all formulas F, G, H, where $\bar{x}$ is the finite list of all free variables in F, G, H:

$$\big((\forall \bar{x}.\,(F \to G)) \wedge (\forall \bar{x}.\,(G \to H))\big) \to (\forall \bar{x}.\,(F \to H)).$$

As an inference rule, with all formulas universally quantified,

$$\frac{F \to G \qquad G \to H}{F \to H}.$$

Expressing $\to$ using $\vee$ and $\neg$ gives $\neg F \vee G$, $\neg G \vee H \vdash \neg F \vee H$. Taking G to be a literal L, and letting $\neg F$ and H be equivalent to clauses C_1 and C_2, we obtain the (simple, propositional-style) resolution rule without unification:

$$\frac{C_1 \vee L \qquad \neg L \vee C_2}{C_1 \vee C_2}$$

Resolution with instantiation

Definition 2.3.11: Complementary literals

Literals L and $\neg L$ are called *complementary*.

If two literals are not complementary, we may still be able to apply the instantiation rule to make them complementary. This gives the general resolution rule

$$\frac{C_1' \vee L' \qquad \neg L'' \vee C_2''}{C_1 \vee C_2}$$

where $C_1 \vee L$ is an instance of $C_1' \vee L'$, $\neg L \vee C_2$ is an instance of $\neg L'' \vee C_2''$ (obtained by instantiating the two clauses so that the literals become identical), and the conclusion is $C_1 \vee C_2$.
To check whether a substitution exists making L' and L'' identical, we use (syntactic, first-order) *unification*, which has a linear-time algorithm. The procedure is:

1. rename the variables in the two clauses so that they are disjoint;
2. compute a *most general unifier* so that the literals become complementary (one an atomic formula, the other its negation);
3. instantiate both clauses with it and apply the resolution rule.

Chapter 3

Quantifier Elimination with Presburger Arithmetic

3.1 Verification Conditions and One-Point Rule

Formal verification methodology

Formal verification of programs can be viewed as a three-step approach:

1. Express properties in logic or code (assertions, preconditions, postconditions, invariants, run-time error conditions).
2. Compile the program's meaning to logical formulas (verification-condition generation, symbolic execution).
3. Develop and use an automated theorem prover for the generated conditions (SAT and SMT solving, resolution-based theorem proving, proof assistants).

The question is which logic to use. Here we study integer linear arithmetic, i.e. Presburger arithmetic.

Presburger arithmetic

Presburger arithmetic is integer arithmetic with the logical operations (and, or, not), quantifiers, only addition as an arithmetic operation, and $<$ and $=$ as relations. Minimalistically, a variant over the non-negative integers can be defined with $\wedge, \neg, \forall, +, =$ as the only symbols.

Use in verification

For a loop with invariant I, the verification condition for preservation of the invariant has the form

$$[\, I(res, i) \wedge i' = i - 1 \wedge res' = res + 2 \wedge 0 < i \,] \rightarrow I(res', i').$$

To prove that such a condition is valid, we check whether its negation is satisfiable, i.e. whether the corresponding existentially quantified Presburger arithmetic formula is true.

One-point rule

Definition 3.1.1: One-point rule

Let $\bar{y}$ be a tuple of variables not containing x. Then

$$\exists x.\, (x = t(\bar{y}) \wedge F(x, \bar{y})) \iff F(t(\bar{y}), \bar{y}).$$

Proof sketch. ($\Rightarrow$) If some value x_1 satisfies $x_1 = t(\bar{y}) \wedge F(x_1, \bar{y})$, then since x_1 and $t(\bar{y})$ have the same value, $F(t(\bar{y}), \bar{y})$ holds. ($\Leftarrow$) If $F(t(\bar{y}), \bar{y})$ holds, take x to be the value of $t(\bar{y})$; then $x = t(\bar{y})$ and $F(x, \bar{y})$ both hold, so the existential holds.

The rule replaces the left-hand side by the right-hand side, eliminating the quantified variable. The reverse direction, used when t is complex, is called *flattening*.

Definition 3.1.2: Dual one-point rule

For universal quantification,

$$\forall x.\,(x = t(\bar{y}) \to F(x, \bar{y})) \iff F(t(\bar{y}), \bar{y}).$$

It follows from the rule for $\exists$ by negating both sides.

Applied repeatedly to the negated verification condition, the one-point rule substitutes away the variables defined by equalities. In the loop case this leaves a formula that simplifies to *false*, so the verification condition is valid. The one-point rule is only one of many steps used in quantifier elimination procedures.

3.2 Sufficient Condition for Quantifier Elimination

Definition 3.2.1: Quantifier elimination

Given a formula $F(\bar{y})$ containing quantifiers, *quantifier elimination* (QE) finds a formula $G(\bar{y})$ that

- is equivalent to $F(\bar{y})$, i.e. for all $\bar{y}$ the two formulas have the same truth value,
- has no quantifiers, and
- has a subset (or equal set) of the free variables of F.

Here $\bar{y}$ is a possibly empty tuple of variables. A theory *admits* QE if such a G exists for every F.

Since G is equivalent to F, we can use it instead of F, and quantifier-free formulas are easier to check for satisfiability.

Consequence for decision procedures

If F has no free variables, then the result of QE has no free variables (they are a subset of the original, empty set) and no quantified variables (it has no quantifiers). It is therefore a formula without any variables, and its truth value is found by simply evaluating it.
Hence, if a theory admits QE:

- To check satisfiability of $H(\bar{y})$, eliminate the quantifiers from $\exists \bar{y}.\, H(\bar{y})$ and evaluate.
- To check validity of $H(\bar{y})$, eliminate the quantifiers from $\forall \bar{y}.\, H(\bar{y})$ and evaluate.

This also lets us decide formulas with arbitrary quantifier structure.

Choice of symbols

Whether Presburger arithmetic admits QE depends on the set of symbols. If we lack operations that can be expressed using quantifiers, there may be no equivalent quantifier-free formula; for instance $\exists y.\, x = y + y + y$ forces us to include divisibility. QE says that if a relation $r \stackrel{def}{=} \{(x, y) \mid F(x, y)\}$ is definable by an arbitrary, possibly quantified, formula, then it is also definable by a quantifier-free formula.

Definition 3.2.2: Presburger arithmetic (PA)

The theory of integers with addition, over the language with the following symbols.

- A constant for each integer constant.
- $+$, $-$, $=$, and $<$ (to compare values and restrict them to natural numbers when needed).
- For each $K \in \mathbb{Z}$ a function m_K, interpreted as multiplication by the constant K, i.e. $m_K(x) = K \cdot x$. There is no multiplication between variables.
- For each K a divisibility predicate $K \mid y$, meaning $y \,\%\, K = 0$. This enables quantifier elimination from $\exists x.\, y = K \cdot x$.

Formulas are built with $\wedge, \vee, \neg, \rightarrow, \exists, \forall$.

Reduction to eliminating a single existential

Nested existentials. A block $\exists x_1.\exists x_2.\exists x_3.\, F_0$ with F_0 quantifier-free is handled by eliminating the innermost quantifier first: eliminating x_3 gives a quantifier-free F_1, then eliminating x_2 gives F_2, and so on, until a quantifier-free $F_3(\bar{y})$ remains.
Universal quantifiers. Since F_0 universally holds iff it has no counterexample,

$$\forall x.\, F_0(x, \bar{y}) \iff \neg[\exists x.\, \neg F_0(x, \bar{y})].$$

As $\neg F_0$ is quantifier-free, we can eliminate $\exists x.\, \neg F_0(x, \bar{y})$, obtaining some quantifier-free $F_1(\bar{y})$, and the result is $\neg F_1(\bar{y})$.
Alternation. Quantifier alternations (switches between $\exists$ and $\forall$) are removed by rewriting every $\forall$ as $\neg\exists\neg$ and eliminating quantifiers from the innermost one outwards. Each alternation involves a disjunctive normal form transformation; in practice there are not many alternations.

Conclusion. It suffices to be able to eliminate a single existential quantifier from a quantifier-free formula.

3.3 DNF transformation

Definition 3.3.1: Atomic formula, literal, conjunction of literals

An *atomic formula* is a relation applied to arguments. In PA the atomic formulas are $t_1 = t_2$, $t_1 < t_2$ and $K \mid t$. A *literal* is an atomic formula or its negation. A *conjunction of literals* is a formula $L_1 \wedge \ldots \wedge L_n$ with no disjunctions or implications, and with negation applied only to atomic formulas.

It suffices to eliminate $\exists y.\, L_1 \wedge \ldots \wedge L_n$ from conjunctions of literals, because any quantifier-free formula F (built from $\wedge, \vee, \neg$ over atomic formulas) can be converted to disjunctive normal form

$$F \iff \bigvee_{i=1}^{m} C_i,$$

where each C_i is a conjunction of literals, and existential quantification distributes over disjunction:

$$\left[\exists x.\, \bigvee_{i=1}^{m} C_i\right] \iff \bigvee_{i=1}^{m} (\exists x.\, C_i).$$

How the DNF transformation works

- **Negation propagation** uses the rewrites $\neg(p \wedge q) \rightsquigarrow (\neg p) \vee (\neg q)$, $\neg(p \vee q) \rightsquigarrow (\neg p) \wedge (\neg q)$ and $\neg\neg p \rightsquigarrow p$. The result is in *negation normal form* (NNF), and the transformation is polynomial.
- **Distributivity** uses $a \wedge (b_1 \vee b_2) \rightsquigarrow (a \wedge b_1) \vee (a \wedge b_2)$. This can lead to exponential blow-up, and the blow-up cannot be avoided in general: one can prove that for some formulas no equivalent DNF of smaller size exists (this is unrelated to the question of NP versus P).

3.4 Eliminating Existentials from Conjunctions of PA Literals

We now consider $\exists y.\, L_1 \wedge \ldots \wedge L_n$ where each L_i is a PA literal.

3.4.1 Normalizing literals

For integers, $\neg(x < y) \iff y \le x$, $x < y \iff x + 1 \le y$, and $x \le y \iff x < y + 1$. We use only $<$ (never $\le$, $>$).

Definition 3.4.1: Linear term

All terms are built from constants K, $+$, $-$ and $K \cdot _$, so by standard transformations they can be written as $K_0 + \sum_{i=1}^{n} K_i x_i$. Such a term is called a *linear term*.

Literals are brought into normal form by the following rewrites.

$$\begin{aligned}
\neg(t_1 < t_2) &\rightsquigarrow t_2 < t_1 + 1 \\
\neg(t_1 = t_2) &\rightsquigarrow t_1 < t_2 \vee t_2 < t_1 \\
t_1 = t_2 &\rightsquigarrow t_1 < t_2 + 1 \wedge t_2 < t_1 + 1 \\
\neg(K \mid t) &\rightsquigarrow \bigvee_{i=1}^{K-1} K \mid t + i \\
t_1 < t_2 &\rightsquigarrow 0 < t_2 - t_1
\end{aligned}$$

The disjunctions this introduces are removed by computing the DNF again. (The rewrite of equalities is used only for simplicity; in practice equalities are usually handled directly.) After normalization every literal has the form $0 < t$ or $K \mid t$ with t linear.
The one-point rule alone is not enough: we must handle inequalities, not just equalities, and over the integers we cannot always divide exactly, so the variable to eliminate may occur as, e.g., $3y$ rather than y.

Note:

In all linear terms, it's possible to group all occurances of a variable into a single occurance.

3.4.2 Exposing the variable: making the coefficient ± 1

We eliminate y from a conjunction of literals of the form $0 < t$ and $K \mid t$. The following equivalences hold for any positive integer c:

- $0 < t$ is equivalent to $0 < c\,t$;
- $K \mid t$ is equivalent to $cK \mid c\,t$.

Let $K_1, \ldots, K_n$ be all coefficients next to y in the formula and let $M = \operatorname{lcm}(K_1, \ldots, K_n)$ (or any positive integer divisible by every K_i). Multiply each literal in which y occurs in a subterm $K_i y$ by $M/|K_i|$. Then every occurrence of y has coefficient M or $-M$, so the formula has the form $\exists y.\, F(M \cdot y)$. Letting $x = My$, this is equivalent to

$$\exists x.\, F(x) \wedge (M \mid x),$$

where the coefficient next to x is now 1 or -1.

3.4.3 Lower and upper bounds

For each literal $0 < t$, if the coefficient of x is -1 move x to the left side, and if it is 1 move the remaining terms to the left side. We obtain a formula $F_1(x)$ of the form

$$\bigwedge_{i=1}^{L} a_i < x \;\wedge\; \bigwedge_{j=1}^{U} x < b_j \;\wedge\; \bigwedge_{i=1}^{D} K_i \mid (x + t_i),$$

where a_i, b_j, t_i do not contain x (the constraint $M \mid x$ is among the divisibility constraints).
No divisibility constraints ($D = 0$). The formula is satisfiable iff $\max_i a_i + 1 \le \min_j b_j - 1$, which is equivalent to

$$\bigwedge_{i,j} a_i + 1 < b_j.$$

Note:
This is only true if $M = 1$ and $D = 0$ initially.

3.4.4 Replacing the variable by test terms

An alternative common technique in quantifier elimination is to replace $\exists x.\, F_1(x)$ by a disjunction $\bigvee_k F_1(t_k)$ of instances at *test terms* t_k not containing x. If some $F_1(t_k)$ holds then certainly $\exists x.\, F_1(x)$ holds.

- If $D = 0$ and $L > 0$, it suffices to try the smallest value allowed by each lower bound:

  $$\bigvee_{k=1}^{L} F_1(a_k + 1),$$

  since one of these terms evaluates to $\max_i a_i + 1$.

- If $D > 0$, let $N = \mathrm{lcm}(K_1, \ldots, K_D)$. Then

  $$\bigvee_{k=1}^{L} \bigvee_{i=1}^{N} F_1(a_k + i)$$

  is equivalent to $\exists x.\, F_1(x)$. Correctness of this reduction relies on the fact that if $F_1(u)$ holds then so does $F_1(u - N)$, as long as the lower bounds still hold. Hence any solution can be moved down to one within N of the largest lower bound.

3.4.5 No lower bounds ($L = 0$)

When there are no lower bounds, we cannot try the lower bounds. The formula has the form

$$\bigwedge_{j=1}^{U} x < b_j \;\wedge\; \bigwedge_{i=1}^{D} K_i \mid (x + t_i).$$

Because x can be made arbitrarily small, we may drop all constraints except the divisibility constraints, obtaining

$$F_2(x) = \bigwedge_{i=1}^{D} K_i \mid (x + t_i),$$

and the quantifier is eliminated as

$$\bigvee_{i=1}^{N} F_2(i).$$

3.4.6 Summary of the algorithm and its consequences

Together, these steps give a quantifier elimination algorithm for Presburger arithmetic. Its correctness shows that:

- PA admits quantifier elimination;
- satisfiability, validity, entailment and equivalence of PA formulas are decidable;
- quantified and quantifier-free PA formulas have the same expressive power.

The algorithm can be used to prove verification conditions. Even though it is not the most efficient approach, it provides insights on which better algorithms can be built.

3.4.7 Improving efficiency in practice

- **Cooper's algorithm** avoids converting to conjunctions of literals and works directly on negation normal form. This removes the need for DNF, at the cost of possibly trying irrelevant bounds. Improvements are due to Reddy and Loveland (bounded quantifier alternation); see also Section 7.2 of *The Calculus of Computation* (Bradley and Manna).
- **Handling equalities efficiently:** treat a system of equalities directly, without eagerly introducing inequalities and divisibility constraints, using Euclid's GCD algorithm for solving linear Diophantine equations (Hermite normal form of a matrix).
- **Weak quantifier elimination:** only eliminate alternations of quantifiers.

3.4.8 Complexity

Regardless of the approach, deciding quantified Presburger arithmetic formulas has inherent computational complexity, characterized by Berman using alternating complexity classes.

Definition 3.4.2: Class $STA(s(n), t(n), a(n))$

A set A is in $STA(s(n), t(n), a(n))$ iff there exists a single-tape alternating Turing machine M_i (making both $\wedge$ and $\vee$ choices) that accepts A and runs within space $s(n)$ and time $t(n)$, performing at most $a(n)$ alternations between $\wedge$ and $\vee$ choices.

Deciding a PA formula of size n with m quantifiers is in $STA(*, 2^{n^{O(m)}}, m)$. The complexity is particularly sensitive to the number of quantifiers m.

Chapter 4

Generating Verification Conditions for Imperative Programs

4.1 Programs and Specifications as Formulas

Given a program and a specification, we want to express program correctness as a *verification condition* (VC), a formula implying that the program satisfies the specification, and to implement VC generation as an algorithm. A program fragment can be represented by a formula relating the initial and final state. For a program with variables x, y, the program denotes a relation on pairs of states (initial values (x, y), final values (x', y')), and this relation is described by a formula over the unprimed (initial) and primed (final) variables.
A specification is likewise a formula relating the old and new values. The program satisfies the specification when its relation is a subset of the relation described by the specification:

$$\{(\bar{x}, \bar{x}') \mid R(c)\} \subseteq \{(\bar{x}, \bar{x}') \mid \mathit{Spec}(\bar{x}, \bar{x}')\}.$$

This subset statement reduces to the validity of the implication

$$R(c) \longrightarrow \mathit{Spec}(\bar{x}, \bar{x}').$$

Thus both programs and specifications are formulas, and correctness is the validity of a formula, which can be checked by a theorem prover.

4.2 Rules for Simple Imperative Programs

Definition 4.2.1: Simple imperative programs

Let F range over formulas and t over terms (with only pure mathematical operations). A program has a fixed number of mutable variables $V = \{x_1, \ldots, x_n\}$. The statements are

- `x = t`: change $x \in V$ to have the value of t, leaving the variables in $V \setminus \{x\}$ unchanged;
- `if(F) c1 else c2`: if F holds execute c_1, else execute c_2;
- `c1 ; c2`: first execute c_1, then c_2.

To introduce and restrict non-determinism there are also

- `havoc(x)`: non-deterministically change $x \in V$ to an arbitrary value, leaving $V \setminus \{x\}$ unchanged;
- `if(*) c1 else c2`: arbitrarily choose to run c_1 or c_2;
- `assume(F)`: block all executions in which F does not hold.

Given a loop-free program c with conditionals, the goal is to compute a polynomial-sized formula $R(c)$ of the form $\exists \bar{z}.\, F(\bar{x}, \bar{z}, \bar{x}')$ describing the relation between the initial values $x_1, \ldots, x_n$ and the final values $x'_1, \ldots, x'_n$.

Definition 4.2.2: Relation formula $R(c)$

For an imperative command c with relation $\rho(c)$ between initial and final states, $R(c)$ is a formula over unprimed variables $\bar{x}$ and primed variables $\bar{x}'$ such that

$$\rho(c) = \{(\bar{x}, \bar{x}') \mid R(c)\}.$$

Assignment

$$R(x = t) \;:\; x' = t \;\wedge \bigwedge_{v \in V \setminus \{x\}} v' = v.$$

The formula must explicitly state which variables remain the same (all except x); otherwise those variables would not be constrained by the relation and could take arbitrary values after the command.

Conditional

$$R(\texttt{if(b)}\ \ c_1\ \ \texttt{else}\ \ c_2) \;:\; (b \wedge R(c_1)) \vee (\neg b \wedge R(c_2)).$$

Sequential composition

Sequencing $c_1; c_2$ corresponds to relation composition:

$$r_1 \circ r_2 = \{(\bar{x}, \bar{x}') \mid \exists \bar{x}''.\, (\bar{x}, \bar{x}'') \in r_1 \wedge (\bar{x}'', \bar{x}') \in r_2\}.$$

Both $R(c_1)$ and $R(c_2)$ are expressed in terms of $\bar{x}$ and $\bar{x}'$. Let $r_i = \{(\bar{x}, \bar{x}') \mid R(c_i)\}$. Then $(\bar{x}, \bar{x}'') \in r_1$ iff $R(c_1)[\bar{x}' := \bar{x}'']$ and $(\bar{x}'', \bar{x}') \in r_2$ iff $R(c_2)[\bar{x} := \bar{x}'']$, so

$$R(c_1; c_2) \iff \exists \bar{x}''.\, R(c_1)[\bar{x}' := \bar{x}''] \wedge R(c_2)[\bar{x} := \bar{x}''],$$

where $\bar{x}''$ are freshly picked names for the intermediate state. A useful convention is to let the names of intermediate variables refer to positions in the program source code.
The resulting existentials can be simplified with the *one-point rule* of logic with equality:

$$(\exists u.\, (u = t \wedge F)) \iff F[u := t] \qquad \text{if } u \notin FV(t).$$

Havoc

$$R(\texttt{havoc}(x)) \;:\; \bigwedge_{v \in V \setminus \{x\}} v' = v.$$

Destroying the value of a variable is described by simply not mentioning the primed variable x', so that it is unconstrained.

Non-deterministic choice

$$R(\texttt{if(*)}\ \ c_1\ \ \texttt{else}\ \ c_2) \;:\; R(c_1) \vee R(c_2).$$

The translation is simply a disjunction, which is why the construct is interesting; it corresponds to branching in control-flow graphs.

Assume

$$R(\texttt{assume}(F)) \;:\; F \wedge \bigwedge_{v \in V} v' = v.$$

This command does not change any state. If F does not hold, it stops with "instantaneous success" (no execution continues), so such executions impose no obligation.

Summary

Definition 4.2.3: Diagonal relation and state set

For a formula F, $S(F) = \{\bar{x} \mid F\}$ denotes the set of states satisfying F. For a set A of states, $\Delta_A = \{(\bar{x}, \bar{x}) \mid \bar{x} \in A\}$ is the *diagonal relation* on A. Without a subscript, Δ is the identity relation on the entire set of states (a no-op).

command c	$R(c)$	$\rho(c)$
$x = t$	$x' = t \wedge \bigwedge_{v \in V \setminus \{x\}} v' = v$	$\{(\bar{x}, \bar{x}') \mid x' = t \wedge \bigwedge_{v \neq x} v' = v\}$
$c_1; c_2$	$\exists \bar{z}.\, R(c_1)[\bar{x}' := \bar{z}] \wedge R(c_2)[\bar{x} := \bar{z}]$	$\rho(c_1) \circ \rho(c_2)$
`if(*)` c_1 `else` c_2	$R(c_1) \vee R(c_2)$	$\rho(c_1) \cup \rho(c_2)$
`assume`(F)	$F \wedge \bigwedge_{v \in V} v' = v$	$\Delta_{S(F)}$

We always have $\rho(c) = \{(\bar{x}, \bar{x}') \mid R(c)\}$.

Definition 4.2.4: Shorthands

The non-deterministic choice `if(*)` c_1 `else` c_2 is written c_1 [] c_2 (a choice between c_1 and c_2), and `assume`(F) is written $[F]$. With these,

$$\texttt{if } (F)\ c_1\ \texttt{else}\ c_2 \equiv ([F]; c_1)\ [] \ ([\neg F]; c_2), \qquad \texttt{if } (F)\ c \equiv ([F]; c)\ [] \ [\neg F].$$

4.3 Loop-Free Programs

Let c be a loop-free program whose assignments, havocs and assumes are $c_1, \ldots, c_n$. Its relation $\rho(c)$ has the form $E(\rho(c_1), \ldots, \rho(c_n))$: it composes the meanings of $c_1, \ldots, c_n$ using only union ($\cup$) and composition ($\circ$). A conditional `if (F)` c_1 `else` c_2 contributes $\Delta_{S(F)} \circ \rho(c_1) \ \cup \ \Delta_{S(\neg F)} \circ \rho(c_2)$. Here $\circ$ binds stronger than $\cup$, so $r \circ s \cup t = (r \circ s) \cup t$.

Normal form

Composition distributes through union:

$$(r_1 \cup r_2) \circ (s_1 \cup s_2) = r_1 \circ s_1 \cup r_1 \circ s_2 \cup r_2 \circ s_1 \cup r_2 \circ s_2.$$

Applying this repeatedly (for instance to two conditionals in sequence) rewrites the relation of any loop-free program as a union of sequential compositions of basic statements.

Definition 4.3.1: Basic path

A sequential composition of basic statements (assignments, havocs, assumes) is called a *basic path*.

Loop-free code thus describes finitely many, but in general exponentially many, basic paths. This is why the polynomial-sized formula $R(c)$, with its intermediate existentially quantified variables, is preferable to enumerating paths.

4.4 Hoare Triples

Strength of conditions

Let P_1 and P_2 be formulas ("conditions") whose free variables are among $\bar{x}$, which may denote program state.

Definition 4.4.1: Stronger and weaker condition

Condition P_1 is *stronger* than P_2 (and P_2 is *weaker* than P_1) iff $\forall \bar{x}.\, (P_1 \rightarrow P_2)$. If we know P_1 we immediately conclude P_2, but knowing P_2 does not allow us to conclude P_1.

A stronger condition denotes a smaller set: if P_1 is stronger than P_2 then $\{\bar{x} \mid P_1\} \subseteq \{\bar{x} \mid P_2\}$. The strongest possible condition, *false*, denotes the smallest set $\emptyset$, and the weakest condition, *true*, denotes the set of all tuples. Weakest conditions correspond to the largest sets, and strongest conditions to the smallest sets, satisfying a given property.

Hoare logic

Hoare logic is a proof system for inserting annotations (assertions) into code to make proofs about imperative program behavior simpler. The assertions are written in braces between statements, and for loops include a loop invariant. We use the relations from the previous sections as program meaning.

Definition 4.4.2: Hoare triple

Let $P, Q \subseteq S$ be sets of states and $r \subseteq S \times S$ a relation. The *Hoare triple* is

$$\{P\}\ r\ \{Q\} \iff \forall s, s' \in S.\ (s \in P \land (s, s') \in r \to s' \in Q).$$

Here $\{P\}$ and $\{Q\}$ do not denote singleton sets; they are just notation for assertions.

Definition 4.4.3: Strongest postcondition

For $P \subseteq S$ and $r \subseteq S \times S$,

$$sp(P, r) = \{s' \mid \exists s.\ s \in P \land (s, s') \in r\}.$$

This is simply the relation image of the set P.

Definition 4.4.4: Weakest precondition

For $Q \subseteq S$ and $r \subseteq S \times S$,

$$wp(r, Q) = \{s \mid \forall s'.\ (s, s') \in r \to s' \in Q\}.$$

This is in general not the same as $sp(Q, r^{-1})$ when the relation is non-deterministic or partial.

Lemma (three forms of a Hoare triple). The following three conditions are equivalent:

- $\{P\}\ r\ \{Q\}$;
- $P \subseteq wp(r, Q)$;
- $sp(P, r) \subseteq Q$.

Proof. The three conditions expand into

$$\begin{aligned}
&\forall s, s'.\ [(s \in P \land (s, s') \in r) \to s' \in Q],\\
&\forall s.\ [s \in P \to (\forall s'.\ (s, s') \in r \to s' \in Q)],\\
&\forall s'.\ [(\exists s.\ s \in P \land (s, s') \in r) \to s' \in Q],
\end{aligned}$$

which are easily shown equivalent using basic first-order logic properties: $(P \land Q \to R) \iff (P \to (Q \to R))$; $(\forall u.\ (A \to B)) \iff (A \to \forall u.\ B)$ when $u \notin FV(A)$; and $(\forall u.\ (A \to B)) \iff ((\exists u.\ A) \to B)$ when $u \notin FV(B)$.

Chapter 5

Abstract Interpretation

5.1 Programs as Control Flow Graphs

Definition 5.1.1: Abstract interpretation

Abstract interpretation is a way to infer properties of program computations. An ordinary interpreter executes a statement on one concrete state. An *abstract interpreter* executes it on an *abstract state*, which represents a set of concrete states (for example, a range of possible values for each variable) and so describes many executions at once. (Cousot and Cousot, POPL 1977.)

A program can be represented as a *control-flow graph* (CFG): vertices are program points, and edges are labelled by statements, either assignments or conditions $[p]$ (assume statements coming from `if` and `while`). An edge without a label is denoted `skip`.

Sets of states at each program point

Given the initial states, the exact meaning of the program can be described by the set of states that can reach each program point. One way to obtain them is to run the CFG from each initial state and insert the resulting states at the appropriate points.
For a command c labelling an edge, let $\rho(c)$ denote the relation between initial and final state that gives the meaning of c.

Definition 5.1.2: Transfer function

The *transfer function* $T(S, c)$ is the image of the set of states S under the relation $\rho(c)$. In general it is defined using the strongest postcondition,

$$T(S, c) = sp(S, \rho(c)).$$

For a condition $[p]$ we have $T(S, [p]) = \{x \in S \mid p\}$, and for an unlabelled edge $T(S, skip) = S$.

Reachable states as a system of equations

The sets of reachable states $S(v)$ for the vertices v of the CFG are described by recursive equations. The entry vertex has the set of all possible initial states, and every other vertex is the union, over its incoming edges (u, c, v), of $T(S(u), c)$:

$$S(v) = \bigcup_{(u,c,v)} T(S(u), c).$$

The solution we want is the unique *least* solution of these equations. It can be computed by iterating the equations starting from empty sets as the initial solution.
The problem is that these exact equations are as difficult to solve as running the program on all possible input states. Instead, we consider approximate descriptions of these sets of states.

5.2 Approximating Sets

Domains of intervals

A large analysis domain is the set of all integer intervals $\{x \mid L \le x \wedge x \le U\}$ for $L, U \in \mathbb{Z}$. It has infinitely many elements, but is already an approximation of all possible sets of integers.
A smaller domain allows only sets of the form $[L, U]$ denoting $\{x \mid L \le x \wedge x \le U\}$ with $L \le U$, where the bounds come from a fixed finite set of constants including `MININT` and `MAXINT`. Then:

- `[MININT,MAXINT]` denotes all integers and is written $\top$;
- all empty intervals are written $\bot$;
- the domain D has only finitely many elements.

Abstract equations

Because only finitely many sets are available, we must approximate, and we approximate sets by possibly larger sets. The equations are rewritten with abstract values $S^\#(v) \in D$, abstract transfer functions $T^\#(S, c)$ giving results in D, and the operation $\sqcup$ in place of $\cup$:

$$S^\#(v) = \bigsqcup_{(u,c,v)} T^\#(S^\#(u), c).$$

Here $S_1 \sqcup S_2$ denotes the approximation of $S_1 \cup S_2$: the set that contains both S_1 and S_2, belongs to D, and is otherwise as small as possible. For intervals,

$$[a, b] \sqcup [c, d] = [\min(a, c), \max(b, d)].$$

Solving the equations

We solve the equations by starting in the initial state and repeatedly applying them: at the entry point we put $\top$, and at all other vertices $\bot$. We iterate until nothing changes, i.e. until a *fixpoint* is found. Mapping the resulting intervals to sets gives a solution of the original constraints (an over-approximation of the reachable states).

Abstract domains are partial orders

Program semantics is given by sets (such as sets of reachable states), which are compared with the subset relation $\subseteq$ and combined with union (for instance at the join after an `if`). To approximate them we introduce a domain of elements $d \in D$, each representing a set.

Definition 5.2.1: Concretization function

The function γ maps an abstract element $d \in D$ to the set of concrete states $\gamma(d)$ it represents. It is called the *concretization function*.

Given d_1, d_2 there need not be an element d with $\gamma(d_1) \cup \gamma(d_2) = \gamma(d)$. Instead we use an element that approximates the union, denoted $d_1 \sqcup d_2$. This leads us to review the theory of partial orders and (semi)lattices.

5.3 Partial Orders and Extreme Elements

Partial orders

Definition 5.3.1: Partial order

A *partial ordering relation* is a binary relation $\leq$ that is, for all x, y, z,

- reflexive: $x \leq x$;
- antisymmetric: $x \leq y \wedge y \leq x \rightarrow x = y$;
- transitive: $x \leq y \wedge y \leq z \rightarrow x \leq z$.

If A is a set and $\leq$ a binary relation on A with these properties, the pair $(A, \leq)$ is called a *partial order*.

Given $\leq$, the corresponding *strict* order $x < y$ is the shorthand for $x \leq y \wedge x \neq y$. Orders on integers, rationals and reals are special cases called *linear orders*. Given a set U and any set $A \subseteq 2^U$ of subsets of U, $(A, \subseteq)$ is a partial order.
A partial order can be drawn as a *Hasse diagram*, a directed graph in the plane in which the direction of an edge is given by which node is drawn above, and transitive and reflexive edges are not drawn (they can be derived).

Extreme elements

Definition 5.3.2: Extreme elements

Let $(A, \leq)$ be a partial order and $S \subseteq A$. An element $a \in A$ is

- an *upper bound* of S if $a' \leq a$ for all $a' \in S$;
- a *lower bound* of S if $a \leq a'$ for all $a' \in S$;
- a *minimal element* of S if $a \in S$ and there is no $a' \in S$ with $a' < a$;
- a *maximal element* of S if $a \in S$ and there is no $a' \in S$ with $a < a'$;
- the *greatest element* of S if $a \in S$ and $a' \leq a$ for all $a' \in S$;
- the *least element* of S if $a \in S$ and $a \leq a'$ for all $a' \in S$;
- the *least upper bound* (lub, supremum, join, $\sqcup$) of S if a is the least element of the set of all upper bounds of S;
- the *greatest lower bound* (glb, infimum, meet, $\sqcap$) of S if a is the greatest element of the set of all lower bounds of S.

Taking $S = A$ gives the minimal, maximal, greatest and least elements of the entire partial order.

Notes:

- a minimal element need not exist (e.g. the open interval $(0, 1)$ of rationals);
- there may be several minimal elements (e.g. in $\{\{a\}, \{b\}, \{a, b\}\}$);
- if a minimal element exists, it need not be least;
- there are no two distinct least elements of the same set;
- the least element is always a glb and minimal;
- if the glb belongs to the set, it is always least and minimal;
- on a family of sets closed under $\cap$ and $\cup$, the glb for the order $\subseteq$ is $\cap$ and the lub is $\cup$; not all families of sets are closed, but the family of all subsets and the family of open sets from topology are.

Least upper bound

Definition 5.3.3: Least upper bound

The *least upper bound* $lub(S)$ of S is the element M, if it exists, that is the least element of the set $U = \{x \mid x \text{ is an upper bound of } S\}$. That is, M is an upper bound of S, and for every other upper bound M' of S we have $M \leq M'$. This is the same definition as the supremum in real analysis.

We write $a_1 \sqcup a_2$ for $lub(\{a_1, a_2\})$, and $(\ldots(a_1 \sqcup a_2)\ldots) \sqcup a_n$ is in fact $lub(\{a_1, \ldots, a_n\})$. The binary operation $\sqcup$ is therefore associative, commutative and idempotent. The greatest lower bound $\sqcap$ is defined dually.

Order on the interval domain

The domain of intervals is $D = \{\bot\} \cup \{(L, U) \mid L \in \{-\infty\} \cup \mathbb{Z},\ U \in \{+\infty\} \cup \mathbb{Z}\}$ with $L \leq U$, and its elements denote sets via

$$\gamma : D \to 2^{\mathbb{Z}}, \qquad \gamma((L, U)) = \{x \mid L \leq x \wedge x \leq U\}.$$

The order is induced by γ: $d_1 \sqsubseteq d_2 \leftrightarrow \gamma(d_1) \subseteq \gamma(d_2)$, hence

$$\begin{gathered}(L_1, U_1) \sqsubseteq (L_2, U_2) \leftrightarrow L_2 \leq L_1 \wedge U_1 \leq U_2, \\ \bot \sqsubseteq d \quad \text{for all } d \in D, \\ (L_1, U_1) \sqcup (L_2, U_2) = (\min(L_1, L_2), \max(U_1, U_2)).\end{gathered}$$

Lenma 5.3.1 Orders from inverse images

Let $\gamma : D \to C$ where C is a collection of sets, and define $d_1 \sqsubseteq d_2 \iff \gamma(d_1) \subseteq \gamma(d_2)$. Then

1. $\sqsubseteq$ is reflexive;
2. $\sqsubseteq$ is transitive;
3. $\sqsubseteq$ is antisymmetric if and only if γ is injective.

If $\sqsubseteq$ is not antisymmetric, we can define the equivalence relation $d_1 \sim d_2 \iff \gamma(d_1) = \gamma(d_2)$ and take D' to be the set of equivalence classes. This is why all empty intervals ($L > U$) are identified and represented by the single element $\bot$.

5.4 Lattices

Definition 5.4.1: Lattice

A *lattice* is a partial order in which every two-element set has a least upper bound and a greatest lower bound. Hence $\sqcap$ and $\sqcup$ are well-defined binary operations.

Lenma 5.4.1 Finite sets in a lattice

In a lattice every non-empty finite set has a lub ($\sqcup$) and a glb ($\sqcap$).

Proof: By induction on the size of the set. For a three-element set $S = \{x, y, z\}$, let $a = (x \sqcup y) \sqcup z$. By definition of $\sqcup$, $z \sqsubseteq a$ and $x \sqcup y \sqsubseteq a$, and also $x \sqsubseteq x \sqcup y$ and $y \sqsubseteq x \sqcup y$, so by transitivity $x \sqsubseteq a$ and $y \sqsubseteq a$; thus a is an upper bound of S. If a' is any upper bound of S, then $x, y \sqsubseteq a'$ gives $x \sqcup y \sqsubseteq a'$, and with $z \sqsubseteq a'$ we get $(x \sqcup y) \sqcup z \sqsubseteq a'$. So a is the least upper bound. The general step is the same, and the argument for $\sqcap$ is dual.

Lenma 5.4.2 Linear orders

Every linear order is a lattice.

Further facts about lattices:

- Every bounded subset of the reals has a lub; this is an axiom of the real numbers (as constructed from the rationals).
- If a lattice has a least and a greatest element, then every finite set (including the empty set) has a lub and a glb.
- This does not imply the existence of lub and glb for infinite sets. For example, $([0,1),\leq)$ is a lattice, but the entire set has no lub in it; the rationals in $[0,10]$ form a lattice, but $\{x \mid 0 \leq x \wedge x^2 < 2\}$ has no lub there.

Lenma 5.4.3 Absorption

In every lattice, $x \sqcup (x \sqcap y) = x$.

Proof: Trivially $x \sqsubseteq x \sqcup (x \sqcap y)$. For the converse, x is an upper bound of both x and $x \sqcap y$, and $x \sqcup (x \sqcap y)$ is the *least* upper bound of these two elements, hence $x \sqcup (x \sqcap y) \sqsubseteq x$.

Definition 5.4.2: Distributive lattice

A lattice is *distributive* iff

$$x \sqcap (y \sqcup z) = (x \sqcap y) \sqcup (x \sqcap z),$$
$$x \sqcup (y \sqcap z) = (x \sqcup y) \sqcap (x \sqcup z).$$

The lattice of all subsets of a set is distributive, and every linear order is a distributive lattice.

Products of lattices

For $n = 2$, a function $f : \{1,2\} \to (L_1 \cup L_2)$ with $f(1) \in L_1$, $f(2) \in L_2$ is isomorphic to the ordered pair $(f(1), f(2))$; the product is denoted $(L_1, \leq_1) \times (L_2, \leq_2)$, and the order is componentwise. Lattice elements can thus be combined into finite or infinite-dimensional vectors, and the result is again a lattice.

Lenma 5.4.4 Product of partial orders and lattices

Let $(A_1, \leq_1), \ldots, (A_n, \leq_n)$ be partial orders. Define

$$A = \{f \mid f : \{1, \ldots, n\} \to (A_1 \cup \ldots \cup A_n) \text{ where } \forall i.\, f(i) \in A_i\},$$

and for $f, g \in A$ let $f \leq g \iff \forall i.\, f(i) \leq_i g(i)$. Then $(A, \leq)$ is a partial order, denoted $\prod_{i=1}^{n} (A_i, \leq_i)$. Moreover, if each $(A_i, \leq_i)$ is a lattice, then $(A, \leq)$ is also a lattice, with $\sqcup$ and $\sqcap$ computed componentwise.

This is a special case of the Feferman–Vaught theorem.

Properties of $\sqcup S$ and $\sqcap S$

Consider a partial order $(A, \sqsubseteq)$.

- If $S_1 \subseteq S_2 \subseteq A$ and $\sqcup S_1$ and $\sqcup S_2$ exist, then $\sqcup S_1 \sqsubseteq \sqcup S_2$.
- If $S_1 \subseteq S_2 \subseteq A$ and $\sqcap S_1$ and $\sqcap S_2$ exist, then $\sqcap S_2 \sqsubseteq \sqcap S_1$.
- If $\sqcup \emptyset$ exists, it is the least element $\bot$; if $\sqcap \emptyset$ exists, it is the greatest element $\top$. This is because every element is both an upper bound and a lower bound of $\emptyset$: the formulas $\forall x. \forall y \in \emptyset.\, y \sqsubseteq x$ and $\forall x. \forall y \in \emptyset.\, y \sqsupseteq x$ are valid.